RSU HPC AI Data Center

Day 2-1

LLM on Kubernetes

Basic K8s • GPU Scheduling • LLM Deployment • Workshop

rsu-login: 171.102.216.177 | rsu-inference: 171.102.216.178 | 2× H100 NVL 94GB

Course Outline – Day 2-1

01

Basic Kubernetes

Pod, Deployment, Service, NodePort

02

GPU on Kubernetes

NVIDIA Device Plugin, GPU Pod, Resource Limit

03

Deploy LLM Inference on K8s

Ollama + Typhoon2 Deployment, Service

04

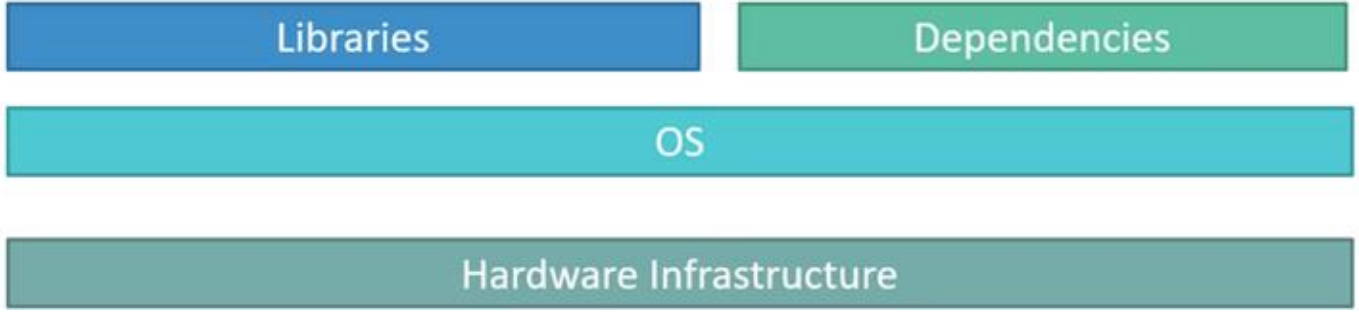
Workshop

Hands-on: Deploy your own LLM Chatbot

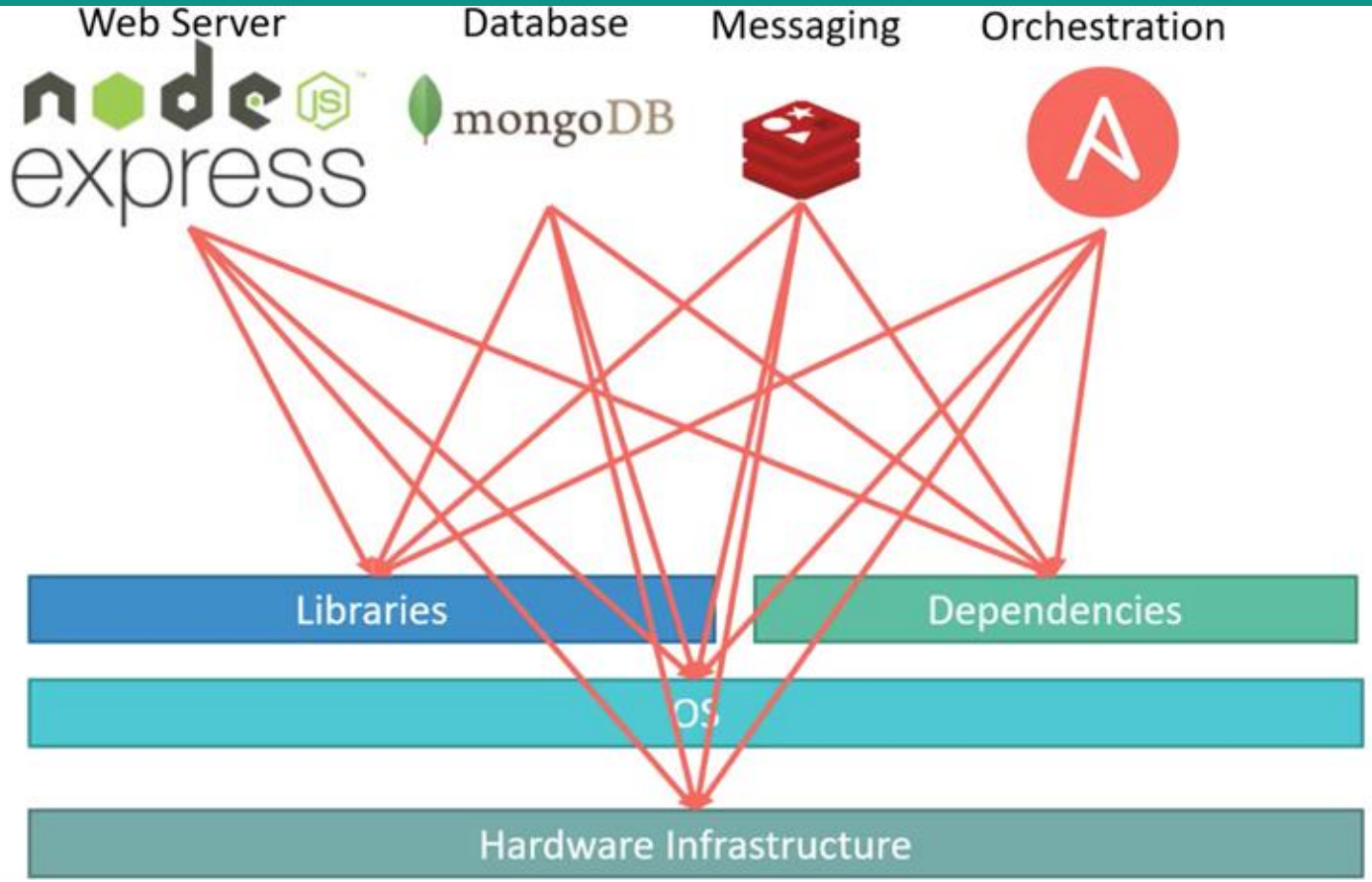
Container overview

Why should we use container? • Container Orchestration

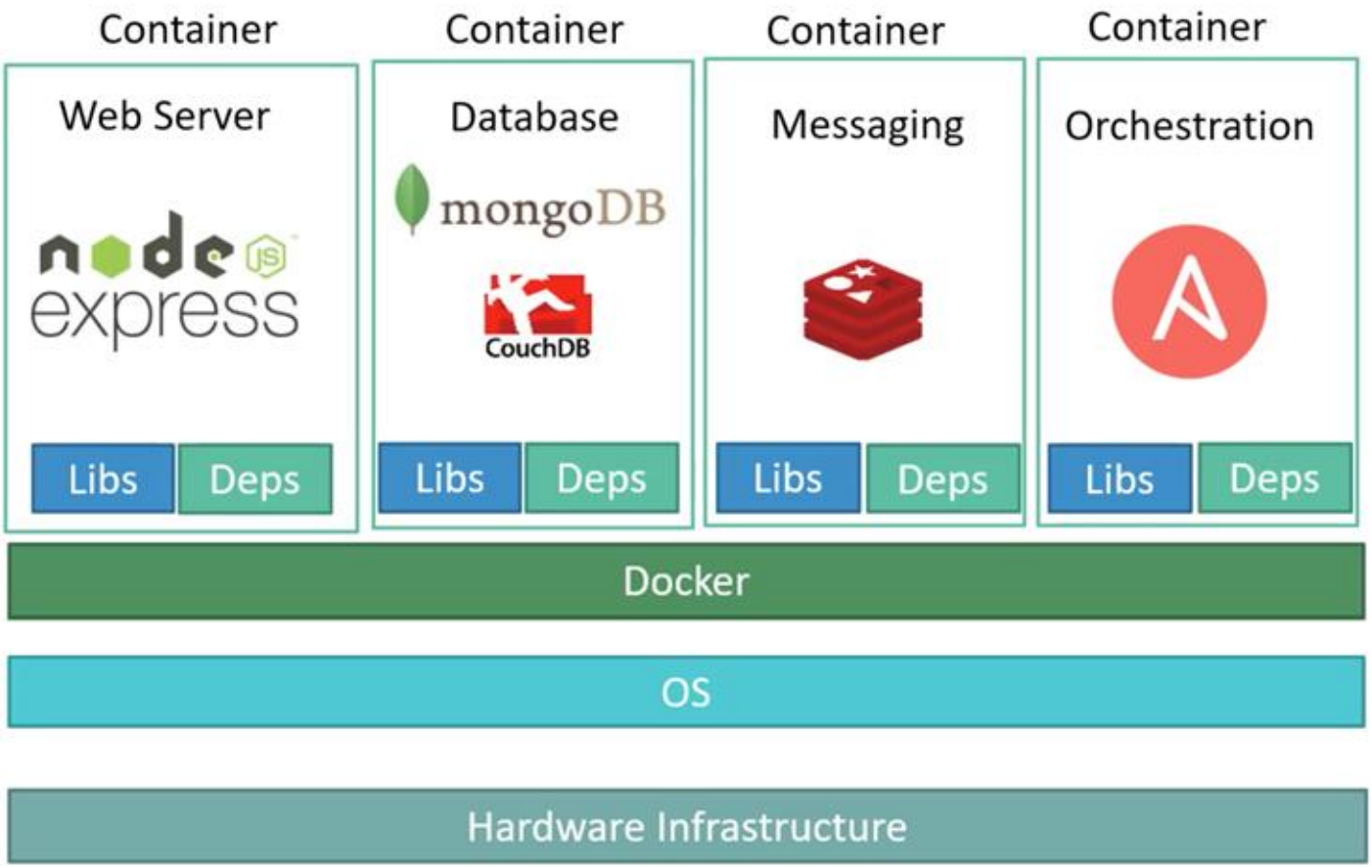
Why should we use container?



Why should we use container?

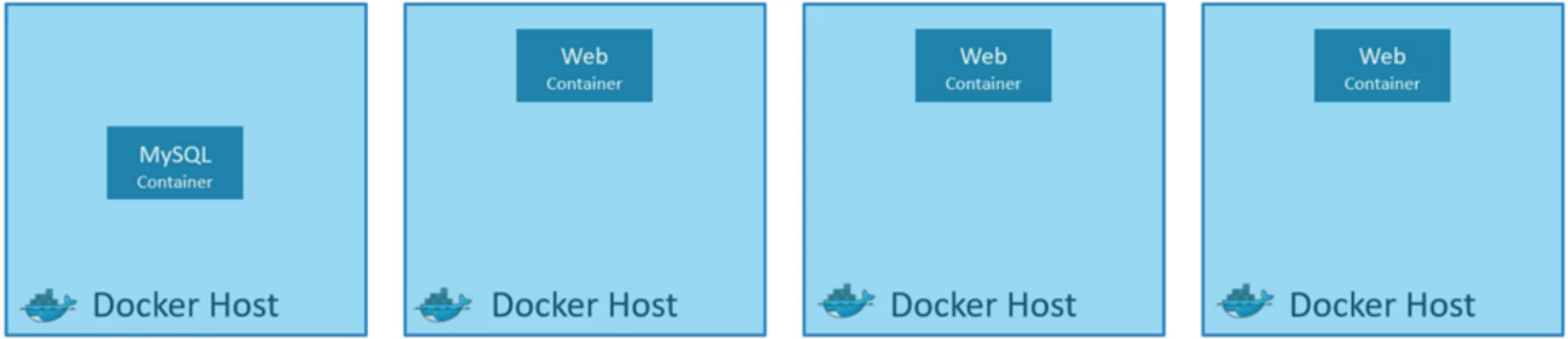


Why should we use container?

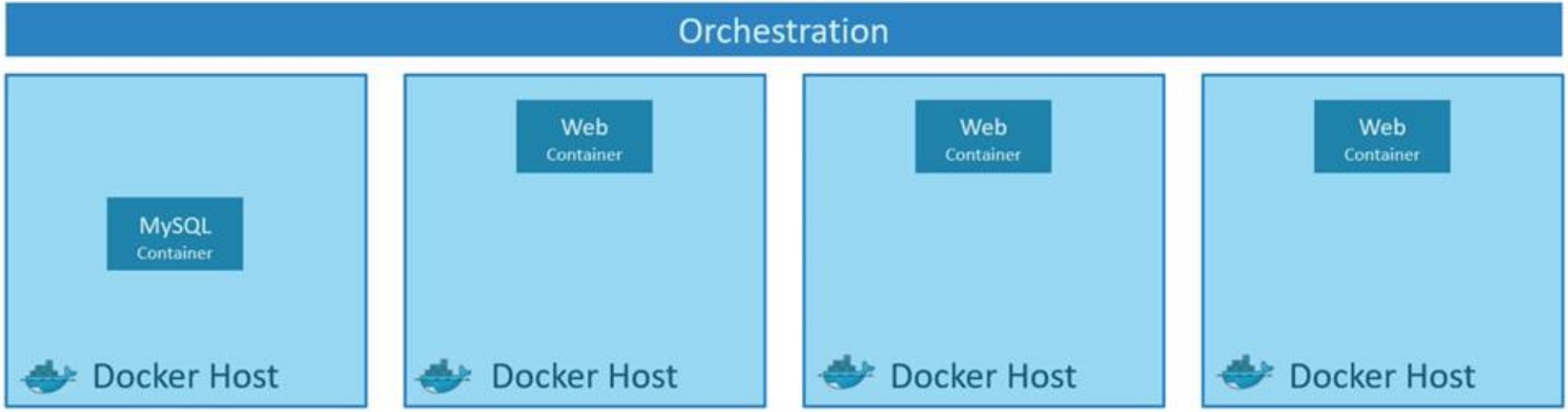


Container Orchestration

Container Orchestration

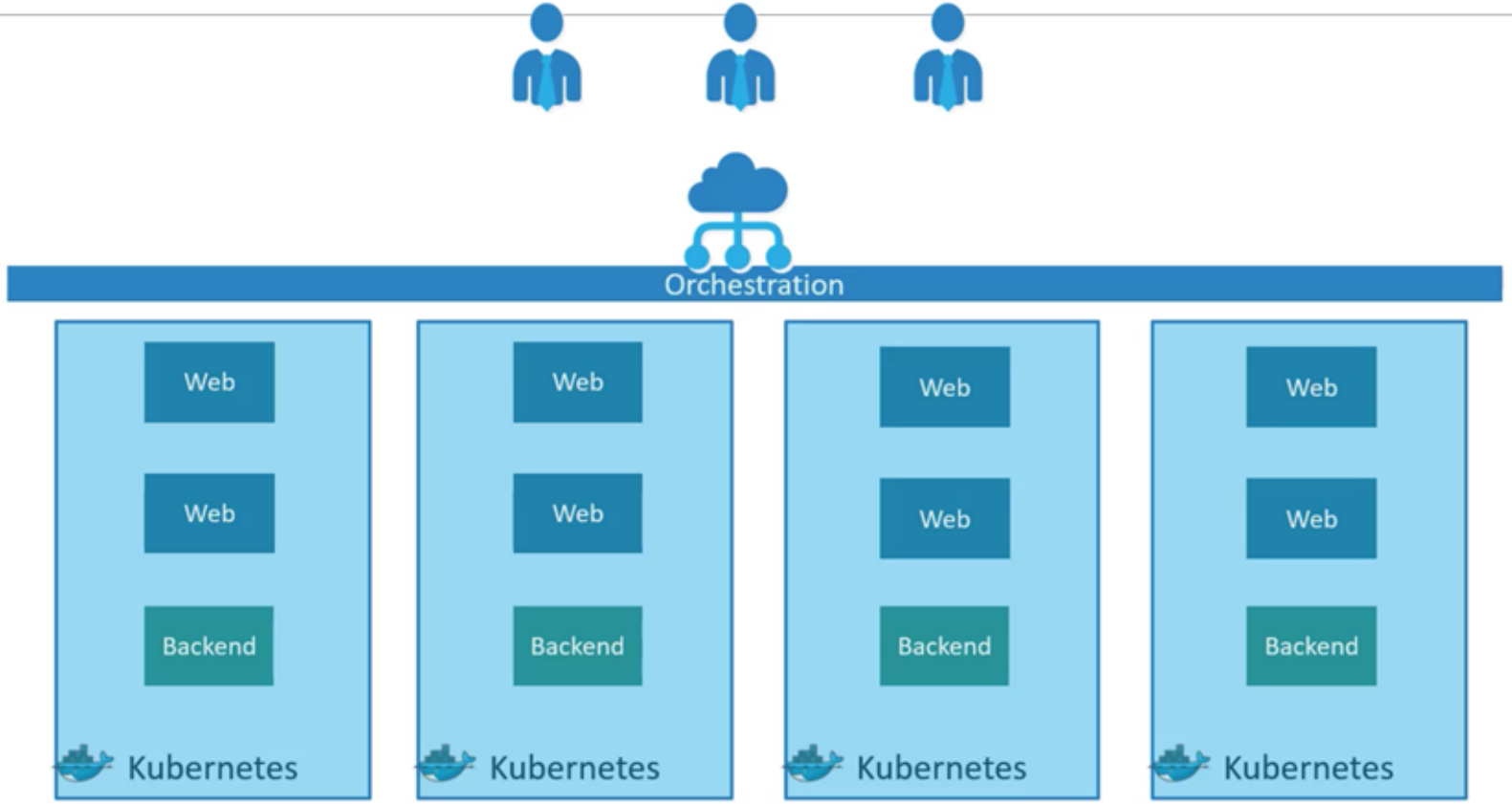


Container Orchestration



Orchestration Technologies

Kubernetes Advantage



Basic Kubernetes

Pod • Deployment • Service • NodePort

Kubernetes คืออะไร?

Kubernetes (K8s) คือระบบ **Container Orchestration** ที่ช่วยจัดการ containers จำนวนมากให้ทำงานอัตโนมัติ

Auto Deploy

Deploy container อัตโนมัติตาม spec

Load Balance

กระจาย traffic ให้ทุก Pod

Self-Healing

Restart Pod ที่ fail อัตโนมัติ

Auto Scale

Scale up/down ตาม workload

RSU Cluster

Master Node (Control Plane)

rsu-login | 32 cores | 251 GiB

Kubernetes API

Worker Node

rsu-inference | 96 cores | 377 GiB

2× NVIDIA H100 NVL 94GB

MIG • 14x 1g.12gb profile

Pod A | **Pod B**

Kubernetes: Concepts หลัก

Pod

หน่วยเล็กที่สุดใน K8s

รัน container 1+ ตัว

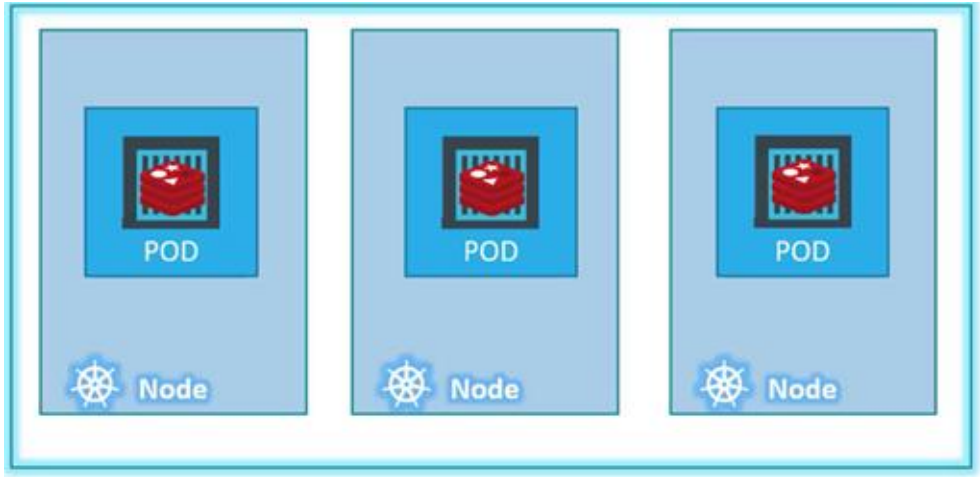
มี IP address ของตัวเอง

pod-definition.yml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end
spec:
  containers:
    - name: nginx-container
      image: nginx
```

```
# create a pod
kubectl create -f pod-definition.yml

# get pod status
kubectl get pods
```



Kind	Version
POD	v1
Service	v1
ReplicaSet	apps/v1
Deployment	apps/v1

Kubernetes: Concepts หลัก

Deployment

จัดการ Pod หลายตัว

กำหนด replicas

Auto-heal เมื่อ Pod ล้ม

deployment-definition.yml

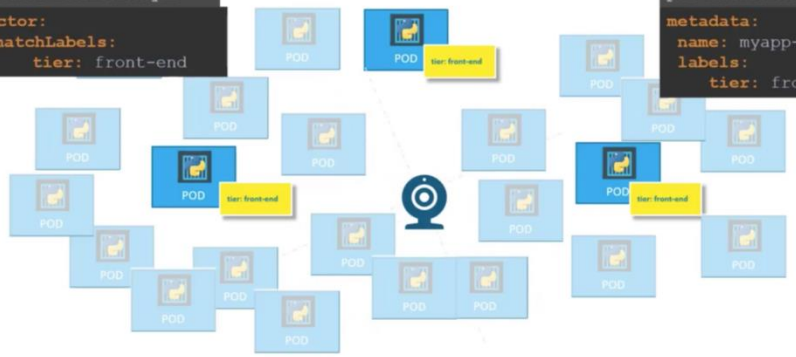
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
  labels:
      app: myapp
      type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
         app: myapp
         type: front-end
    spec:
      containers:
      - name: nginx-container
        image: nginx
  replicas: 3
  selector:
    matchLabels:
       type: front-end
```

replicaset-definition.yml

```yaml
selector:
    matchLabels:
        tier: front-end
```

pod-definition.yml

```yaml
metadata:
  name: myapp-pod
  labels:
     tier: front-end
```




```
> kubectl create -f deployment-definition.yml
deployment "myapp-deployment" created
```

```
> kubectl get deployments
NAME              DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
myapp-deployment   3         3         3            3           21s
```

```
> kubectl get replicaset
NAME                          DESIRED   CURRENT   READY   AGE
myapp-deployment-6795844b58   3         3         3       2m
```

```
> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
myapp-deployment-6795844b58-5rbjl   1/1     Running   0          2m
myapp-deployment-6795844b58-h4w55   1/1     Running   0          2m
myapp-deployment-6795844b58-1fjhv   1/1     Running   0          2m
```

Kubernetes: Concepts หลัก

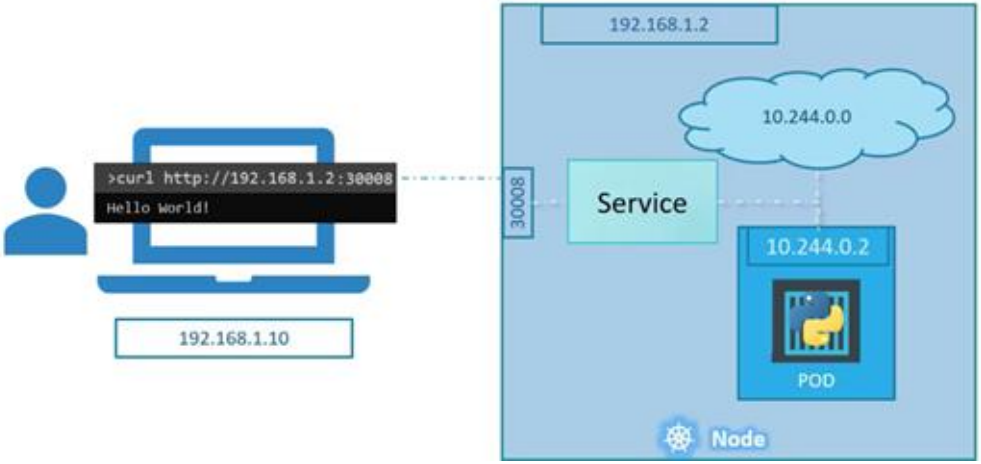
Service / NodePort

Expose Pod ออกภายนอก
NodePort: 30000-32767
สามารถเข้าผ่าน NodeIP:Port

service-definition.yml

```yaml
apiVersion: v1
kind: Service
metadata:
    name: myapp-service

spec:
    type: NodePort
    ports:
     - targetPort: 80
       port: 80
       nodePort: 30008
    selector:
       app: myapp
       type: front-end
```



Kubernetes: Concepts หลัก

Service / NodePort

Expose Pod ออกภายนอก

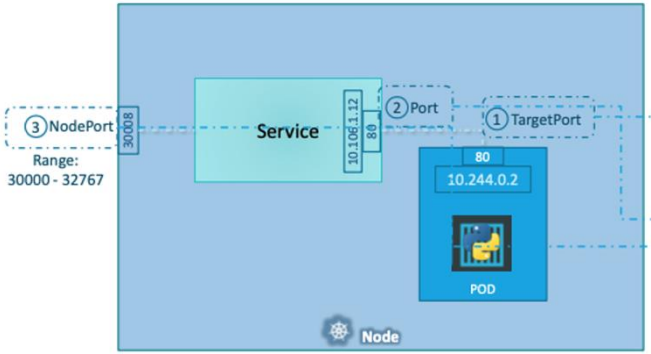
NodePort: 30000-32767

สามารถเข้าผ่าน NodeIP:Port

service-definition.yml

```yaml
apiVersion: v1
kind: Service
metadata:
    name: myapp-service

spec:
    type: NodePort
    ports:
     - targetPort: 80
       port: 80
       nodePort: 30008
    selector:
       app: myapp
       type: front-end
```



service-definition.yml

```yaml
apiVersion: v1
kind: Service
metadata:
    name: myapp-service

spec:
    type: NodePort
    ports:
     - targetPort: 80
       port: 80
       nodePort: 30008
```

Kubernetes: Concepts หลัก

Service / NodePort

Expose Pod ออกภายนอก

NodePort: 30000-32767

สามารถเข้าผ่าน NodeIP:Port

service-definition.yml

```yaml
apiVersion: v1
kind: Service
metadata:
    name: myapp-service

spec:
    type: NodePort
    ports:
     - targetPort: 80
       port: 80
       nodePort: 30008
    selector:
      app: myapp
      type: front-end
```

pod-definition.yml

```yaml
apiVersion: v1
kind: Pod

metadata:
 name: myapp-pod
 labels:
    app: myapp
    type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx
```

Kubernetes: Concepts หลัก

Service / NodePort

Expose Pod ออกภายนอก

NodePort: 30000-32767

สามารถเข้าผ่าน NodeIP:Port

service-definition.yml

```yaml
apiVersion: v1
kind: Service
metadata:
    name: myapp-service

spec:
    type: NodePort
    ports:
     - targetPort: 80
       port: 80
       nodePort: 30008
    selector:
       app: myapp
       type: front-end
```

```
> kubectl create -f service-definition.yml
service "myapp-service" created
```

```
> kubectl get services
NAME            TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
kubernetes      ClusterIP   10.96.0.1        <none>        443/TCP        16d
myapp-service   NodePort    10.106.127.123   <none>        80:30008/TCP   5m
```

```
> curl http://192.168.1.2:30008
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
</style>
</head>
<body>
```

kubectl – คำสั่งพื้นฐาน

```
kubectl get pods
```

ดู Pod ทั้งหมด

```
kubectl get pods -n <namespace>
```

ดู Pod ใน namespace ที่กำหนด

```
kubectl apply -f pod.yaml
```

สร้าง resource จากไฟล์ YAML

```
kubectl delete pod <name>
```

ลบ Pod

```
kubectl logs <pod-name>
```

ดู logs ของ Pod

```
kubectl exec -it <pod> -- bash
```

เข้า shell ใน Pod

```
kubectl describe pod <name>
```

ดูรายละเอียด Pod

```
kubectl get services
```

ดู Services ทั้งหมด

Namespace – แยกพื้นที่ของแต่ละ User

ใน RSU Cluster แต่ละ user จะมี namespace เป็นของตัวเองเพื่อป้องกันการชนกัน

RSU Kubernetes Cluster

ns: user01-restricted

myapp-pod
port: 11434

NodePort Svc
30434, 30435...

ns: user02-restricted

myapp-pod
port: 11434

NodePort Svc
30434, 30435...

ns: user03-restricted

myapp-pod
port: 11434

NodePort Svc
30434, 30435...

คำสั่งที่ใช้บ่อย:

```
kubectl get pods -n user01-restricted        kubectl apply -f pod.yaml        kubectl get svc -n user01-restricted
```

GPU on Kubernetes

NVIDIA Device Plugin • GPU Pod • MIG

Kubernetes จัดการ GPU อย่างไร?

1

NVIDIA Device Plugin

DaemonSet ที่รันบนทุก Node มี GPU
ตรวจจับ GPU และแจ้ง Kubernetes scheduler

2

Resource Request

Pod ขอใช้ GPU ผ่าน resources.limits
'nvidia.com/gpu: 1'

3

K8s Scheduler

จัดสรร Pod ไปยัง Node ที่มี GPU ว่าง
ติดตาม GPU allocation อัตโนมัติ

4

Pod ได้ใช้ GPU

Container เข้าถึง GPU ผ่าน CUDA
รัน inference ได้ทันที

GPU Pod YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: cuda-test
  namespace: test01-restricted
spec:
  restartPolicy: Never
  containers:
  - name: cuda-test
    image: nvcr.io/nvidia/k8s/cuda-
sample:vectoradd-cuda11.7.1-ubuntu20.04
    resources:
      limits:
        nvidia.com/gpu: "1"
```

MIG – แบ่ง GPU ให้หลาย User

MIG (Multi-Instance GPU) ช่วยแบ่ง H100 1 ตัว ออกเป็น instances เล็กๆ ได้ถึง 7 ตัว

H100 #0 (94GB)

1g.13gb ~13 GB	1g.13gb ~13 GB	1g.13gb ~13 GB	1g.13gb ~13 GB
1g.13gb ~13 GB	1g.13gb ~13 GB	1g.13gb ~13 GB	

H100 #1 (94GB)

1g.13gb ~13 GB	1g.13gb ~13 GB	1g.13gb ~13 GB	1g.13gb ~13 GB
1g.13gb ~13 GB	1g.13gb ~13 GB	1g.13gb ~13 GB	

188 GB VRAM	14 slots	7g.94gb / 1g.13gb	~14 users (GPU)
2 H100 รวมกัน	MIG instances สูงสุด	Profile ที่ใช้	User พร้อมกัน

ตรวจสอบ GPU Resources ใน K8s

คำสั่งสำหรับดู GPU ที่มีใน cluster และ Pod ที่กำลังใช้งาน

ดู GPU resource บน Node

```
kubectl get node rsu-inference -o json | \
  jq '.status.allocatable' | grep nvidia

# Output:
"nvidia.com/gpu": "14"
```

ดู Pod ที่กำลังใช้ GPU

```
kubectl get node -o custom-columns=\
"NAME:.metadata.name,\
GPU-CAPACITY:.status.capacity.nvidia\.com/gpu,\
GPU-ALLOCATABLE:.status.allocatable.nvidia\.com/gpu”

NAME            GPU-CAPACITY   GPU-ALLOCATABLE
rsu-inference   14             14
rsu-login       <none>         <none>
```

สร้างและรัน GPU test pod

```
# create a pod
kubectl apply -f cuda-test.yaml -n user01-restricted

# ดู status (รอจนเป็น Completed)
kubectl get pod cuda-test -n user01-restricted

# ดู output (จะเห็น nvidia-smi)
kubectl logs cuda-test -n user01-restricted

# delete a pod
kubectl delete -f cuda-test.yaml -n user01-restricted
```

output ที่คาดหวัง

```
[Vector addition of 50000 elements]
Copy input data from the host memory to the CUDA device
CUDA kernel launch with 196 blocks of 256 threads
Copy output data from the CUDA device to the host memory
Test PASSED
Done
pod "cuda-test" deleted
```

Deploy LLM Inference on K8s

Ollama + Typhoon2 • Deployment • Service

Architecture: LLM Chatbot on K8s

Gradio UI

Web Interface

หน้าเว็บสำหรับ
ผู้ใช้งาน
Port 31657

HTTP API

Ollama

LLM Runtime

รัน LLM model
จาก Hugging Face
Port 11434

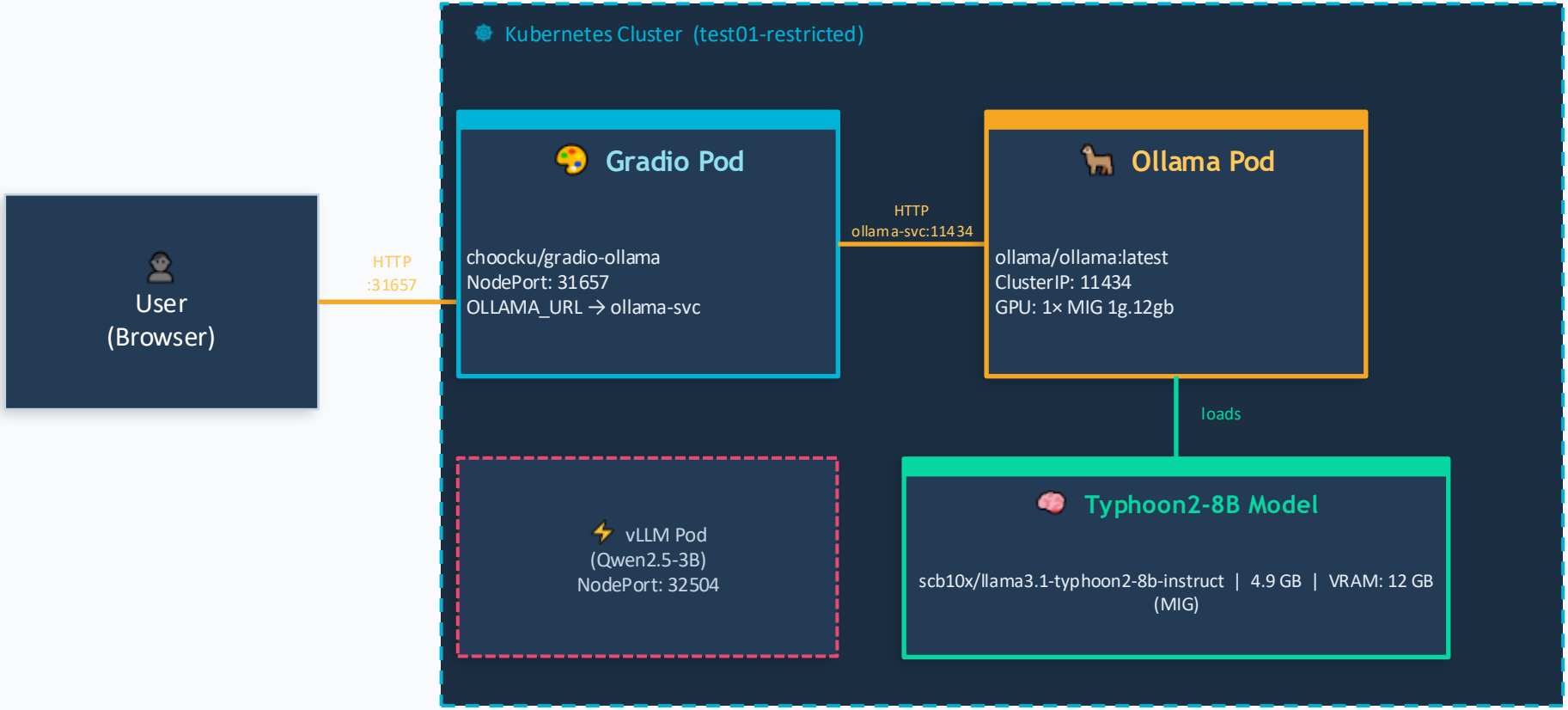
loads

Typhoon2-8B

Thai LLM Model

โมเดลภาษาไทย
จาก SCB 10X
4.9 GB

Architecture: LLM Chatbot on K8s



ขั้นตอนการทำงาน

1

User พิมพ์ข้อความ

ใน Gradio UI
http://171.102.216.177:31633/

2

Gradio ส่ง Request

POST /api/generate
ไปยัง ollama-svc:11434

3

Ollama ประมวลผล

ส่ง prompt ไปยัง
Typhoon2-8B model

4

Streaming Response

ส่ง token กลับ
ทีละ chunk (stream)

5

แสดงผลใน UI

Gradio render
ข้อความแบบ real-time

Ollama Deployment YAML

ollama-deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ollama
  namespace: user01-restricted
spec:
  replicas: 1
  selector:
    matchLabels:
      app: ollama
  template:
    metadata:
      labels:
        app: ollama
    spec:
      containers:
      - name: ollama
        image: ollama/ollama:latest
        ports:
        - containerPort: 11434
        resources:
          limits:
            nvidia.com/gpu: "1"
        env:
        - name: OLLAMA_HOST
          value: "0.0.0.0:11434"
        - name: OLLAMA_MODELS
          value: "/models"
        volumeMounts:
        - name: ollama-models
          mountPath: /models
      volumes:
      - name: ollama-models
        hostPath:
          type: Directory
          path: /data/home/test06/ollama_models/models
```

ollama-service.yaml + Commands

```yaml
# Ollama Service (ClusterIP)
apiVersion: v1
kind: Service
metadata:
  name: ollama-svc
  namespace: user01-restricted
spec:
  selector:
    app: ollama
  ports:
  - port: 11434
    targetPort: 11434
  type: ClusterIP

---

# Deploy command
kubectl apply -f ollama-deployment.yaml

# Pull Typhoon2 model into running pod
kubectl exec -it \
  deployment/ollama -- \
  ollama pull typhoon2-8b-instruct

# Check model loaded
kubectl exec -it \
  deployment/ollama -- \
  ollama list

# Restart Deployment
kubectl rollout restart deployment/ollama
```

Gradio UI Deployment + NodePort

gradio-deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: gradio-chatbot
  namespace: user01-restricted
spec:
  replicas: 1
  selector:
    matchLabels:
      app: gradio
  template:
    metadata:
      labels:
        app: gradio
    spec:
      containers:
      - name: gradio
        image: choocku/gradio-ollama:latest
        ports:
        - containerPort: 7860
        env:
        - name: OLLAMA_URL
          value: "http://ollama-svc:11434"
```

gradio-service.yaml + คำสั่ง

```yaml
apiVersion: v1
kind: Service
metadata:
  name: gradio-svc
  namespace: user01-restricted
spec:
  type: NodePort
  selector:
    app: gradio
  ports:
  - port: 7860
    targetPort: 7860

---

# เข้าถึงจากภายนอกได้ที่:
# http://171.102.216.178:<nodeport>

# Apply ทั้งหมด:
kubectl apply -f gradio-deployment.yaml
kubectl apply -f gradio-service.yaml

# ดู Pod status:
kubectl get pods
kubectl get svc


# Restart Deployment
kubectl rollout restart deployment apps/gradio-chatbot
```

Gradio UI Deployment + NodePort

```
$ kubectl get all -n test01-restricted

NAME                                READY   STATUS    RESTARTS   AGE
pod/gradio-chatbot-87cd66c5d-lwcvm     1/1     Running   0          2m
pod/ollama-98f8f686d-vdhbj             1/1     Running   0          20h


NAME                    TYPE        CLUSTER-IP       PORT(S)
service/gradio-svc       NodePort    10.150.10.54     7860:31657/TCP
service/ollama-svc       ClusterIP   10.150.59.244    11434/TCP
```

Workshop

ลงมือ Deploy LLM Chatbot บน K8s จริง

Workshop Overview

61 คนเรียน | ~14 slots GPU (MIG 1g) | 2 Phase

Phase 1 — CPU Pod – ทุกคนพร้อมกัน

ฝึก kubectl + YAML workflow

1. สร้าง namespace ของตัวเอง
2. kubectl apply pod.yaml (nginx / python)
3. kubectl get pods / logs / exec
4. สร้าง NodePort Service
5. ทดสอบเข้าถึงจาก browser

📌 *ทุก 61 คนทำพร้อมกันได้ ไม่ต้องใช้ GPU*

Phase 2 — GPU Pod – หมุนเวียน 3 รอบ

~14 คน / รอบ • ~20 นาที / รอบ

1. Deploy Ollama pod พร้อม GPU request
2. kubectl exec -- ollama pull typhoon2-8b-instruct
3. ทดสอบ inference ผ่าน REST API
4. Deploy Gradio UI pod
5. เชื่อม Gradio → Ollama (ClusterIP)

📌 *Cleanup ก่อนออก: kubectl delete deployment --all -n <ns>*

Lab Phase 1 – CPU Pod (ทุกคนพร้อมกัน)

Step 1: ตรวจสอบ Namespace ของตนเอง

```
kubectl get all
No resources found in user01-restricted namespace.
```

Step 2: สร้าง Pod แรก (nginx)

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  namespace: user01-restricted
  labels:
    app: myapp
    type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx
```

Step 3: Apply และตรวจสอบ

```
kubectl apply -f pod-definition.yml
kubectl get pod myapp-pod
kubectl describe pod myapp-pod
kubectl logs myapp-pod
```

Step 4: สร้าง NodePort Service

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
  namespace: user01-restricted
  labels:
    app: myapp
spec:
  type: NodePort
  selector:
    app: myapp
    type: front-end
  ports:
    - port: 80
      targetPort: 80
```

Lab Phase 2 – Ollama + Typhoon2 บน GPU

ทำทีละ ~14 คน • รอคิวผ่าน rotation • cleanup ก่อนออกทุกครั้ง

Step 1: Deploy Ollama + GPU

```
kubectl apply -f ollama-deployment.yaml
kubectl apply -f ollama-service.yaml

# รอจนเป็น Running
kubectl get pod -w
```

Step 2: Pull Typhoon2 Model

```
kubectl exec -it deploy/ollama -- ollama
pull scb10x/llama3.1-typhoon2-8b-instruct

# ใช้เวลา ~5-10 นาที
```

Step 3: ตรวจสอบ Model บน Ollama

```
kubectl exec -it deploy/ollama -- ollama list
```

```
(base) [test06@rsu-login ollama+gradio]$ kubectl exec -it deploy/ollama -- ollama list
NAME                                        ID              SIZE      MODIFIED
scb10x/llama3.1-typhoon2-8b-instruct:latest    979081e0492e    4.9 GB    16 seconds ago
```

Step 4: Deploy Gradio + ทดสอบ

```
kubectl apply -f gradio-deployment.yaml
kubectl apply -f gradio-service.yaml

# กรณีที่ไม่เห็น Model ให้ทำการ restart Gradio Deployment
kubectl rollout restart deployment/gradio-chatbot

# ตรวจสอบ Pod, Deployment และ Service
Kubectl get all

# เปิด browser:
# http://171.102.245.177:<nodeport>
```

Step 5: CLEANUP (สำคัญมาก!)

```
# ลบ pod และ deployment ทั้งหมด
kubectl delete deployment --all

# ปล่อย GPU slot ให้คนถัดไป
```

Troubleshooting – ปัญหาที่พบบ่อย

⚠ Pod อยู่ใน Pending

สาเหตุ: GPU ไม่มีว่าง / Resource limit เกิน

แก้ไข:
kubectl describe pod <name> → ดู Events
รอคนอื่น cleanup หรือลด GPU request

⚠ ImagePullBackOff

สาเหตุ: ชื่อ image ผิด หรือ registry ไม่ได้รับอนุญาต

แก้ไข:
ตรวจสอบชื่อ image ใน YAML
ใช้ image จาก docker.io / nvcr.io

⚠ CrashLoopBackOff

สาเหตุ: Container start แล้ว crash ซ้ำๆ

แก้ไข:
kubectl logs <pod> --previous
ดู error message แล้วแก้ YAML

⚠ Service ไม่ตอบสนอง

สาเหตุ: selector ไม่ match label บน Pod

แก้ไข:
ตรวจสอบ app: label บน Pod
ให้ตรงกับ selector ใน Service

สรุป – Day 2-1

- ✅ Kubernetes: Pod → Deployment → Service
- ✅ GPU: NVIDIA Device Plugin + MIG partition บน H100
- ✅ LLM Deploy: Ollama + Typhoon2 บน K8s + NodePort
- ✅ Workshop: CPU pod (ทุกคน) + GPU LLM chatbot (หมุนเวียน)

RSU HPC AI Data Center

rsu-login 171.102.216.177 rsu-inference 171.102.216.178
2× NVIDIA H100 NVL 94GB • 14x MIG 1g.12gb profile • Kubernetes